

MICROSOFT AI INNOVATORS PROJE SUNUMU

GÜLCE TAHTASIZ

PROJE SEÇİMİ

PROJE KONUSU: YELKEN YARIŞLARI İÇİN TAKIM PLANLAMA

PROJE AMACI: Koçların sisteme kaydettiği sporcu verilerini analiz ederek, her rol için en uygun adayları belirlemek ve yarışa en güçlü takım kompozisyonunu oluşturmak



HAFTALIK İLERLEME

1. HAFTA

ML'e Giriş

Temel kavramlar: sınıflandırma (classification), regresyon (regression), kümeleme (clustering).

Amaç: ML nedir, nasıl çalışır; temel iş akışı (veri → model → değerlendirme).

2. HAFTA

Proje Seçimi & Veri Hazırlığı

Problem tanımı ve hedefler.

Veri hazırlığı, model ailesi seçimi ve üstüne yoğunlaşma

3. HAFTA

Model Mimarisi & Eğitim

Mimari: katman sayısı, aktivasyonlar, dropout, sigmoid

Optimizasyon: AdamW/SGD, early stopping.

4. HAFTA

Değerlendirme & Test

Metrikler: doğruluk (accuracy), micro/macro F1, AUC, Hamming.

Threshold ayarlama.

ablation/iyileştirme, sonuçların dokümantasyonu.

PROJEMDE ML NASIL KULLANILIYOR?

Projemde, koların sisteme kaydettiėi sporcu verileriyle ok etiketli (multi-label) bir MLP modeli eėitiyorum. Model her rol iin uygunluk skorları retiyor; ardından Hungarian ataması ile yariř iin en uygun takım kompozisyonuseiliyor.

01

Grev & Mimari

- Grev: 7 rol iin multi-label classification (bařst, direk dibi, piyano, trim-1, trim-2, anayelken, dmenci).
- Girdi (X): cinsiyet, yař, boy, kilo, karar sresi, dikkat, st vcut gc, tutma gc, tm vcut gc(plank saniyesi), eviklik, deneyim yılı.
- ıkıř (Y): her rol iin 0/1 etiket.
- Mimari: MLP (giriř → 64 → 32 → 7), ReLU aktivasyon, Dropout(0.2), ıkıřta sigmoid skorlar.

MODEL A: 128->64->7

```
#MLP MODEL
class MLP(nn.Module):
    def __init__(self, in_dim, out_dim):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(in_dim, 128), nn.ReLU(),
            nn.Linear(128, 64), nn.ReLU(),
            nn.Dropout(0.2),
            nn.Linear(64, out_dim)
        )
    def forward(self, x): return self.net(x)
```

```
[TEST] Subset accuracy (exact match): 0.200
[TEST] Label-wise accuracy: {'label_basustu': 0.733, 'label_direkdi
bel_anayelken': 0.667, 'label_dumen': 0.8}
[TEST] Micro P/R/F1: 0.732 / 0.837 / 0.781
[TEST] Macro P/R/F1: 0.765 / 0.787 / 0.750
[TEST] Per-role F1: {'label_basustu': 0.81, 'label_direkdibi': 0
': 0.737, 'label_dumen': 0.625}
[TEST] Per-role AUC: {'label_basustu': np.float64(0.905), 'label
float64(0.967), 'label_trim2': np.float64(0.735), 'label_anayelk
[TEST] Hamming loss: 0.219
```

MODEL B: 64->32->7

```
#MLP MODEL
class RoleMLP(nn.Module):
    def __init__(self, in_dim, out_dim):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(in_dim, 64), nn.ReLU(),
            nn.Linear(64, 32), nn.ReLU(),
            nn.Dropout(0.2),
            nn.Linear(32, out_dim)
        )
    def forward(self, x): return self.net(x)
```

```
[TEST] Subset accuracy (exact match): 0.133
[TEST] Label-wise accuracy: {'label_basustu': 0.933, 'label_direkdib
bel_anayelken': 0.733, 'label_dumen': 0.8}
[TEST] Micro P/R/F1: 0.752 / 0.837 / 0.792
[TEST] Macro P/R/F1: 0.750 / 0.795 / 0.757
[TEST] Per-role F1: {'label_basustu': 0.944, 'label_direkdibi': 0.54
elken': 0.8, 'label_dumen': 0.625}
[TEST] Per-role AUC: {'label_basustu': np.float64(0.964), 'label_dir
float64(1.0), 'label_trim2': np.float64(0.725), 'label_anayelken': n
[TEST] Hamming loss: 0.205
```

02

Veri Hazırlığı & Eğitim

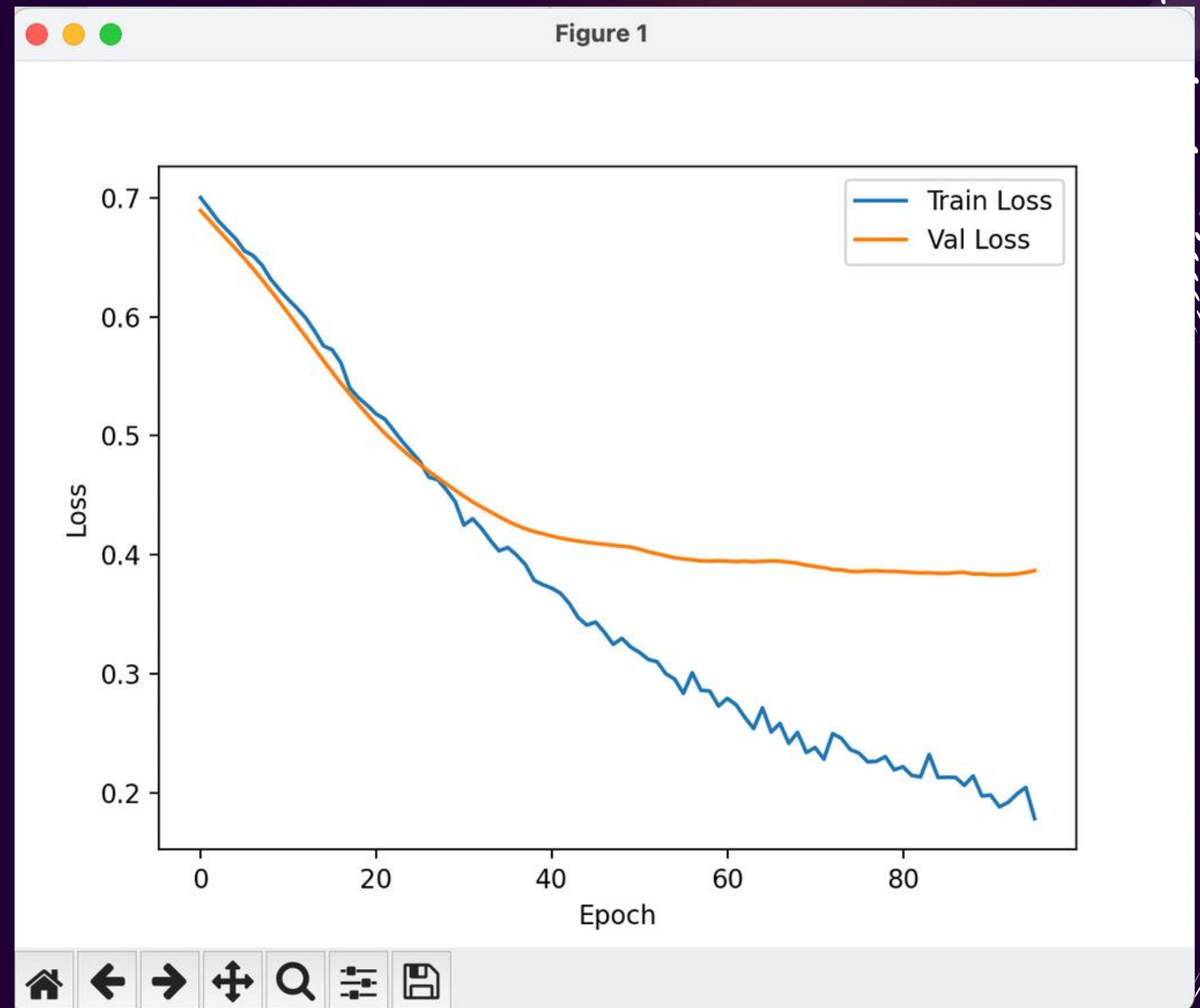
- Veri: tutarlı sentetik veri, 150 satır
- Bölme: train/val/test (60/20/20), StandardScaler.
- Loss/optimizasyon: BCEWithLogitsLoss, AdamW, batch=32, epoch=100, early stopping (patience=5).
- Tekrarlanabilirlik: sabit seed.

```
CSV_PATH = "sailing_150.csv"
EPOCHS   = 100
PATIENCE  = 5
BATCH     = 30
SEED      = 3
```

```
labels = {}
labels["basustu"] = ((df["agility_ms"]<900) & (df["plank_sec"]>100)).astype(int)
labels["direkdibi"] = ((df["upper_watts"]>550) & (df["grip_kg"]>40)).astype(int)
labels["piyano"] = ((df["attention"]>70) & (df["decision_ms"]<400)).astype(int)
labels["train"] = ((df["attention"]>65) & (df["upper_watts"]>50)).astype(int)
```


LOSS/EPOCH

```
Epoch 084 | train_loss=0.2127 | val_loss=0.3843  
Epoch 085 | train_loss=0.2129 | val_loss=0.3843  
Epoch 086 | train_loss=0.2127 | val_loss=0.3849  
Epoch 087 | train_loss=0.2061 | val_loss=0.3851  
Epoch 088 | train_loss=0.2140 | val_loss=0.3838  
Epoch 089 | train_loss=0.1972 | val_loss=0.3838  
Epoch 090 | train_loss=0.1979 | val_loss=0.3831  
Epoch 091 | train_loss=0.1880 | val_loss=0.3831  
Epoch 092 | train_loss=0.1918 | val_loss=0.3833  
Epoch 093 | train_loss=0.1990 | val_loss=0.3839  
Epoch 094 | train_loss=0.2044 | val_loss=0.3849  
Epoch 095 | train_loss=0.1781 | val_loss=0.3866  
Early stop.
```



03 İyileştirme & Değerlendirme

- Threshold ayarı: validation verisinde en iyi F1 değerine göre rol-bazlı threshold seçimi (grid 0.30–0.80). Sonrasında aynı thresholdları test verisinde kullanarak metriklerle doğruluk ölçtüm.
- Metrikler: subset accuracy, micro/macro P-R-F1, label-wise F1, ROC-AUC, Hamming loss test verisiyle gözlemledim.

```
===== Role-based thresholds =====
basustu    -> best_thr=0.35 | best_F1=0.765
direkdibi  -> best_thr=0.35 | best_F1=0.909
piyano     -> best_thr=0.35 | best_F1=0.700
trim1      -> best_thr=0.50 | best_F1=0.857
trim2      -> best_thr=0.30 | best_F1=0.786
anayelken  -> best_thr=0.65 | best_F1=0.815
dumen      -> best_thr=0.35 | best_F1=0.857
```

```
===== TEST metrics (using thresholds chosen on VAL) =====
Subset accuracy (exact match): 0.333
[TEST] Label-wise accuracy: {'basustu': 0.8, 'direkdibi': 0.867, 'piyano': 0.7, 'trim1': 0.967, 'trim2': 0.733,
, 'anayelken': 0.767, 'dumen': 0.9}
Micro Precision: 0.795
Micro Recall: 0.778
Micro F1: 0.787
Macro Precision: 0.851
Macro Recall: 0.757
Macro F1: 0.776
Per-role F1: {'basustu': 0.824, 'direkdibi': 0.6, 'piyano': 0.743, 'trim1': 0.957, 'trim2': 0.733, 'anayelken':
: 0.774, 'dumen': 0.8}
Per-role AUC: {'basustu': np.float64(0.878), 'direkdibi': np.float64(0.988), 'piyano': np.float64(0.818), 'tri
m1': np.float64(0.995), 'trim2': np.float64(0.824), 'anayelken': np.float64(0.912), 'dumen': np.float64(0.942)
}
Hamming loss: 0.181
```

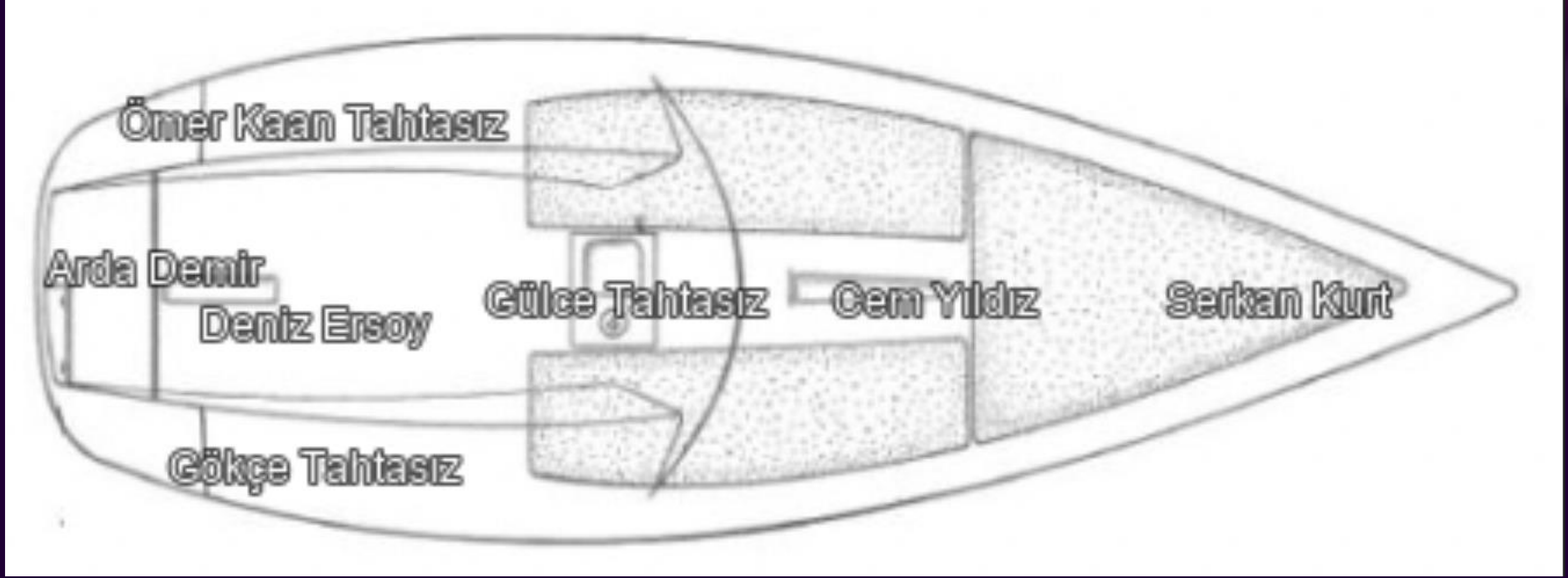

04

Rol Ataması (Hungarian) & Çıktılar

- Amaç: Her role 1 kişi atayarak toplam takım skorunu maksimize etmek.
- Girdi: Modelin tahmin ettiği skor matrisi
- Kısıtlar: Bir kişi en fazla 1 rol, her rol tam 1 kişi.
- Çıktılar:
- scores_from_mlp.csv (tüm skorlar)
- team_selection.csv (role, athlete_id, score)
- Irace_team.png (isimler rollerine göre teknede olmaları gereken yerlere yazılır).

team_selection.csv

```
1 role,athlete_id,score
2 anayelken,Deniz Ersoy,0.9986207485198975
3 basustu,Serkan Kurt,0.9580869674682617
4 direkdişi,Cem Yıldız,0.9268054366111755
5 dumen,Arda Demir,0.9215171933174133
6 piyano,Gölce Tahtasız,0.9840519428253174
7 trim1,Ömer Kaan Tahtasız,0.9988859295845032
8 trim2,Gökçe Tahtasız,0.9995267391204834
9 |
```



**DİNLEDİĞİNİZ İÇİN
TEŞEKKÜRLER!**

KAYNAKÇA

ML EĞİTİMİ & HUNGARIAN ALGORİTMASI

edx- ibm machine learning with python online course

YouTube- StatQuest with Josh Starmer- The Essential Main Ideas of Neural Networks

e- ibm-what are MLPs?

Medium - Afaf Athar - Activation Functions, Optimization Techniques, and Loss Functions

Medium -Yiğit Şener -Makine Öğrenmesinde Train, Validation ve Test Kavramları ile Python Uygulaması

Medium - Sanjay Dutta-How to Calculate Precision, Recall, F1, Confusion Matrix, ROC AUC for Deep Learning Models

Medium - Carolina Bento - Multilayer Perceptron Explained with a Real-Life Example and Python Code: Sentiment Analysis

Brilliant- Hungarian Maximum Matching Algorithm

GeeksForGeeks- Hungarian Algorithm for Assignment Problem (Introduction and Implementation)

ChatGPT

KOD ÖRNEKLERİ

Machine Learning Mastery- Building Multilayer Perceptron Models in PyTorch

GITHUB- GLAZARadr - Multi Layer Perceptron Using Pytorch

ChatGPT